# Amélioration d'un jeu mobile : image matching et recapture

DIARRA Amadou, TOTLADZE David, BAJI Jules
Professeur Encadrant: CLAUX Frédéric

12 janvier 2024

# Table des matières


**I Image matching** **6**

**1 SIFT** **6**

1.1 Invariance au changement d'échelle . . . . . 7

1.2 Sélection des points clés . . . . . 8

1.3 Invariance au changement d'angle . . . . . 9

**2 BRISK** **10**

2.1 Détection de points clés espace échelle . . . . . 10

2.2 Description du point clé . . . . . 11

2.2.1 Création du Descripteur BRISK : Modélisation, Échantillonnage et Construction du Descripteur Binaire . . . . . 12

2.2.2 Correspondance des descripteur . . . . . 13

2.3 Répétabilité . . . . . 13

2.3.1 Exemple de correspondance BRISK . . . . . 13

**3 KAZE** **14**

3.1 Modélisation de la Diffusion Non Linéaire : Concepts Fondamentaux et Approches Numériques . . . . . 14

3.2 Caracteristiques de KAZE . . . . . 15

3.2.1 Calcul de l'espace d'échelle non linéaire . . . . . 15

3.3 Détection des caractéristiques . . . . . 16

3.4 Construction du Descripteur KAZE . . . . . 16

3.4.1 Estimation de l'Orientation et Détection des Points Clés . . . 16

3.4.2 Construction du Descripteur . . . . . 16

II Apprentissage 16

1 Architecture 17

1.1 Réseau de neurones . . . . . 17

1.2 Classificateur . . . . . 18

2 Réseau siamois 18

2.1 Partage des paramètres . . . . . 18

2.2 Fonction de perte . . . . . 19

III Recapture 20

1 Méthode par apprentissage de coefficients de corrélation avec classificateur SVM 20

2 Méthode par apprentissage de descripteurs LTC avec classificateur neuronal 22

# Table des figures


1 Floutage progressive . . . . . 7

2 Différence entre les gaussiennes [4] . . . . . 8

3 Pixels voisins [4] . . . . . 8

4 Voisinage du point clé en rouge [5] . . . . . 9

5 Détection de points d'intérêt à l'espace échelle : Un point clé . . . . . 11

6 Gros plan d'un exemple de détection BRISK sur les images 1 et 2 de la séquence Boat présentant un petit zoom et une rotation dans le plan. 11

7 Le modèle d'échantillonnage BRISK avec N=60 points : les petits cercles bleus indiquent les lieux d'échantillonnage ; les cercles plus grands, en pointillés rouges, sont tracés à un rayon $\sigma$ correspondant à l'écart-type du noyau gaussien utilisé pour lisser les valeurs d'intensité aux points d'échantillonnage.Le modèle présenté s'applique à une échelle de t=1. . . . . 12

8 Exemple de correspondance BRISK . . . . . 14

9 Comparaison entre l'espace d'échelle de diffusion gaussien et non linéaire pour plusieurs temps d'évolution ti . . . . . 15

10 Un réseau [9] . . . . . 17

11 Réseau siamois [3] . . . . . 19

12 Dans la rangée supérieure, une paire d'images capturées et recapturées en niveaux de gris. Dans la rangée du bas, leur version résiduelle. 21

13 Extraction des patchs et calcul des coefficients de corrélation par pixel, en prenant $c_{1,1}$ comme exemple. On conserve le triangle supérieur de la matrice de coefficients. . . . . 22

14 Schéma fonctionnel (a) de la phase d'entraînement et (b) de la phase d'essai. . . . . 22

15 Procédé d'imagerie d'un appareil photo grand public . . . . . 23

16 Calcul du code LTC . . . . . 24

17 Schéma du processus d'extraction des caractéristiques LTC . . . . . 25

# Introduction

L'objectif de notre projet consiste à développer un algorithme d'image matching qui permettra de déterminer si deux images sont approximativement similaires. Ceci sera implémenté dans une application mobile, le but de l'application est de lancer un défi à une personne pour qu'il prenne en photo un objet (ou un animal, etc) demandé. La photo ne peut être prit seulement si l'application détermine que l'objet prit en photo est similaire à l'objet demandé. L'application et des téléphones seront fournis et le langage Python sera utilisé pour le développement.

Mais l'image matching induit une autre problématique, celui de la recapture. Avec les progrès de la technologie d'affichage des images et l'utilisation généralisée de divers dispositifs d'acquisition d'images, il devient très pratique et facile de recapturer des images de haute qualité à partir d'écrans LCD (affichage à cristaux liquides) de haute fidélité. Ces images recapturées constituent une menace sérieuse pour les technologies actuelles de criminalistique d'images et les systèmes de reconnaissance intelligents. Les applications critiques telles que l'identification biométrique des personnes (par exemple, la reconnaissance faciale) sont confrontées à des problèmes d'usurpation de visage, le système pouvant être trompé en montrant la photo d'une autre personne sur un écran électronique.

Dans le domaine de la comparaison d'images, il existe diverses techniques pour évaluer si deux images sont similaires et pour détecter la recapture. Cependant, au sein de ce projet, nous mettons l'accent sur deux familles d'approches, à savoir l'apprentissage et l'analyse de descripteurs.

Pour la problématique de l'image matching, ce rapport vise à fournir un aperçu de trois solutions avec l'analyse de descripteurs : les algorithmes SIFT, BRISK et KAZE. Ainsi qu'une méthode par apprentissage : le réseau siamois. Pour la reconnaissance de recapture, nous verrons les classificateurs binaire et SVM, et l'analyse de descripteurs LTC.

## Première partie

# Image matching

## 1 SIFT

SIFT (Scale-Invariant Feature Transform) est l'une des approches par descripteurs permettant de déterminer si deux images sont approximativement similaires. SIFT a était développé par David G. Lowe en 2004.

L'algorithme SIFT permet d'extraire, d'une image, des points clés et les ca-

ractéristiques qui les définissent. SIFT est aussi invariant au changement d'échelle et à la rotation de l'image sur laquelle il est utilisé. C'est grâce à ces fonctions qu'il est possible de déterminer si deux images sont identiques. Nous allons maintenant voir les différentes étapes du fonctionnement de SIFT.

## 1.1 Invariance au changement d'échelle

L'algorithme doit être invariant au changement d'échelle. C'est-à-dire que si l'on veut déterminer la similitude entre deux photos d'un même objet, mais l'objet ayant une taille un peu différente dans les deux images, il faut que l'algorithme soit capable d'affirmer que ces deux images sont similaires. Pour cela, l'algorithme doit appliquer certaines altérations aux images.

Tout d'abord, l'algorithme applique à une image un échelonnage pour rendre l'image de plus en plus petites. On appelle chaque niveau d'échelle une octave. Les images ainsi créées sont deux fois plus petite que la précédente. Par exemple, si l'image d'origine est de taille 1000x1000 pixels, la première image échelonnée sera de taille 500x500 pixels. La deuxième image échelonnée le sera à partir de la première image créée elle-même échelonnée, donc la taille de la deuxième image sera de 250x250 pixels. Et ainsi de suite jusqu'à une certaine limite qu'on aura définie.

On applique aussi à l'image de chaque octave un filtre gaussien pour flouter l'image. Il faut créer plusieurs images floutées. Le flou doit être progressif pour les images créées, donc on applique plusieurs fois le filtre gaussien à l'image d'origine (voir figure 1) avec un facteur différent.

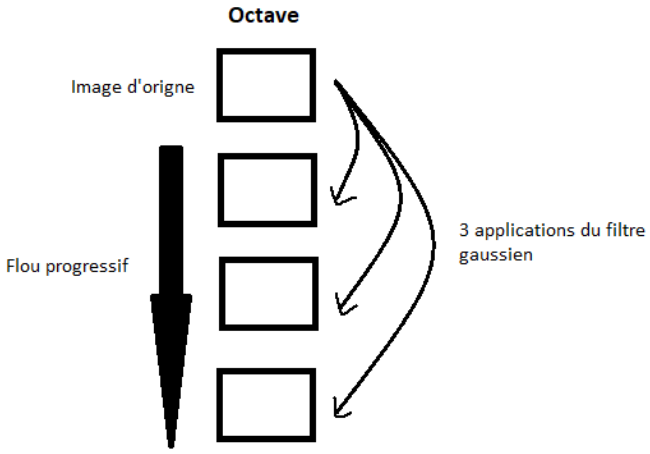



FIGURE 1 – Floutage progressive

Ensuite, on créé une nouvelle liste d'image pour chaque octave en faisant une soustraction entre deux images consécutives d'une octave. Ceci est appelé le calcul de la différence entre les gaussiennes (voir figure 2). Les nouvelles images conservent les caractéristiques de l'image de base qui permettent de trouver des points clés. Grâce à tout cela, SIFT sera invariant au changement d'échelle.

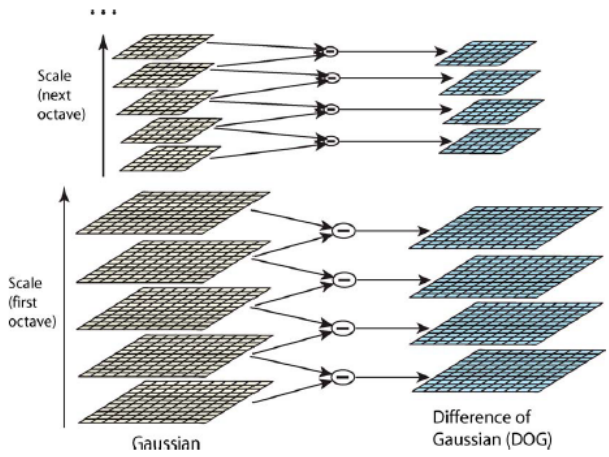



FIGURE 2 – Différence entre les gaussiennes [4]

## 1.2 Sélection des points clés

Maintenant, nous devons commencer la recherche de points clés. C'est grâce à ces points clés que nous qu'il sera possible de déterminer si deux images sont similaires. Les points clés sont des extrema locaux des images. Un extremum local est un pixel qui est plus lumineux ou plus sombre que ses voisins, nous devons donc utiliser la valeur d'intensité des pixels. Pour trouver ces extrema, l'algorithme compare pour chaque pixels ses 26 pixels voisins à travers les différents octaves. Il y a 8 pixels voisins pour l'image du pixel ainsi que les 9 pixels voisins de l'image de l'octave qui se trouve à l'échelle précédente et les 9 pixels voisins de l'image de l'échelle suivante (voir figure 3). Si la valeur de l'intensité d'un pixel est inférieur ou supérieur à tous ses voisins alors c'est un extrema et donc un potentiel point clé.

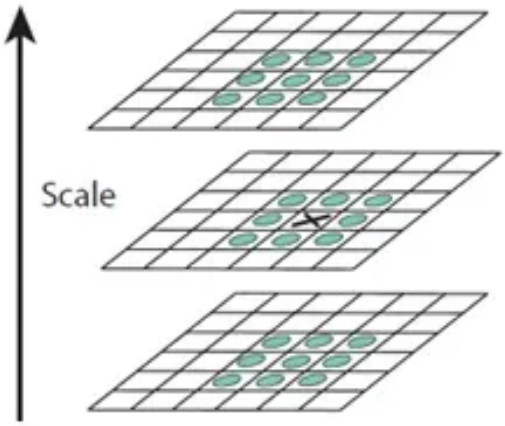



FIGURE 3 – Pixels voisins [4]

Beaucoup de pixels sont trouvés et certains ne sont pas utiles, ils ne correspondent pas aux points clés cherchés. Il faut les trier en se débarrassant de ceux qui n'ont pas assez de contraste ou qui se situent sur les bords puisque ces pixels n'ont pas 26 pixels voisins et donc la comparaison avec les pixels voisins n'a pas fonctionné correctement. Pour le contraste, il faut définir un niveau d'intensité et les pixels qui sont en dessous de ce niveau sont ne seront pas utilisés.

## 1.3 Invariance au changement d'angle

Comme pour le problème du changement d'échelle vu précédemment, il y a aussi un problème de changement d'angle à régler. Si nous prenons en photo un objet et que nous comparons cette photo avec une photo du même objet, mais selon un angle légèrement différent, il faut pouvoir déterminer que ces deux images sont approximativement similaires.

Pour cela, il faut attribuer des orientations aux points clés. Une région autour d'un point clé est définie en fonction de la taille de l'image. Ensuite, dans cette zone, on calcule la magnitude du gradient. La magnitude du gradient d'un pixel d'une image est l'intensité du changement de luminosité (ou intensité) du pixel. Il faut aussi calculer la direction du gradient qui représente l'orientation du changement d'intensité du pixel. La magnitude et la direction du gradient sont calculés grâce aux dérivées partielles horizontales (x) et verticales (y) des pixels.

Puis, il faut déterminer la direction dominante. On utilise un histogramme avec 36 colonnes, une colonne pour chaque 10° (nous pouvons paramétrer le nombre de colonnes et les angles qu'elles représentent). Des valeurs sont assignées aux colonnes en utilisant les valeurs des magnitudes calculées. Par exemple si pour un pixel, on a calculé une direction de 25°, la valeur de la magnitude de ce même gradient sera ajouté dans la colonne qui représente les angles entre 20° et 29°. Ainsi, les colonnes ayant des valeurs supérieures à un certain seuil seront choisies. Les colonnes vont de 10° en 10° alors on attribut au point clé par exemple 25° si une colonne dominante représente les angles entre 20° et 29°.

Ensuite pour chaque point clé, on défini une autre région autour de lui. Cette région est divisée en plusieurs blocs et dans chaque bloc nous faisant la même chose que l'étape précédente (voir figure 4). Tous ces angles ainsi que le ou les angles du point clé déterminés précédemment sont normalisés et mis dans un vecteur qui représente le descripteur du point clé choisi.

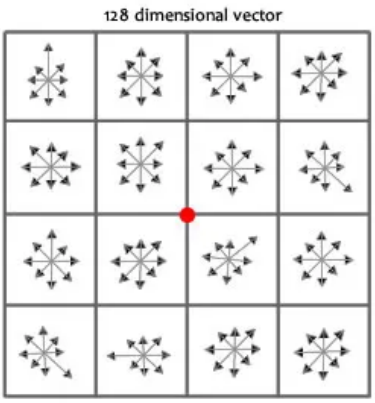



FIGURE 4 – Voisinage du point clé en rouge [5]

Pour finir, pour déterminer si deux images sont identiques, il faut comparer les descripteurs des points clés de ces deux images et si les descripteurs sont similaires (on défini le seuil de similarité) alors les deux images sont similaires.

# 2 BRISK

**BRISK : Binary Robust Invariant Scalable Keypoints** se distingue en tant que descripteur largement utilisé dans le domaine de la détection et de la correspondance d'images. Apparu en 2011 grâce aux travaux de Stefan Leutenegger, Margarita Chli et Roland Y. Siegwart, ce descripteur se caractérise par sa conception visant à être à la fois robuste et scalable. Il offre des performances compétitives dans la détection et la description de points d'intérêt sur des images, même en présence de transformations complexes telles que des changements de perspective, des rotations et des variations d'échelle.
**Fonctionnement de l'algorithme BRISK**
Son fonctionnement peut-être divisé plusieurs points clé :

## 2.1 Détection de points clés espace échelle

Les points clés dans une image sont des positions spécifiques qui représentent des caractéristiques distinctives. L'échelle, dans le contexte de l'image, fait référence à la taille relative des objets ou des structures présentes. Une échelle plus grande permet de capturer des structures plus larges, tandis qu'une échelle plus petite rend le système plus sensible aux détails fins.

L'espace échelle ou **scale space**, est une représentation multi-échelle d'une image. Cette représentation offre la possibilité de considérer l'image à différentes résolutions spatiales, ce qui est crucial pour analyser des détails à différentes échelles.

En ce qui concerne BRISK, l'espace échelle est organisé en couches pyramidales composées d'octaves (**ci**) et d'intra-octaves (**di**), notées pour $i = 0, 1, \ldots, n-1$, avec $n$ typiquement égal à 4. Les octaves sont formées par un demi-échantillonnage progressif de l'image d'origine (correspondant à **c0**). Chaque intra-octave est positionné entre les couches **ci** et **ci**+**1**. Le point clé, qui représente le point le plus remarquable sur l'image, est identifié à une échelle spécifique (appelée octave **ci**). Cette identification se fait en analysant les scores de saillance du point ainsi que de ses voisins dans cette échelle, en tenant compte également des échelles juste au-dessus et en dessous.

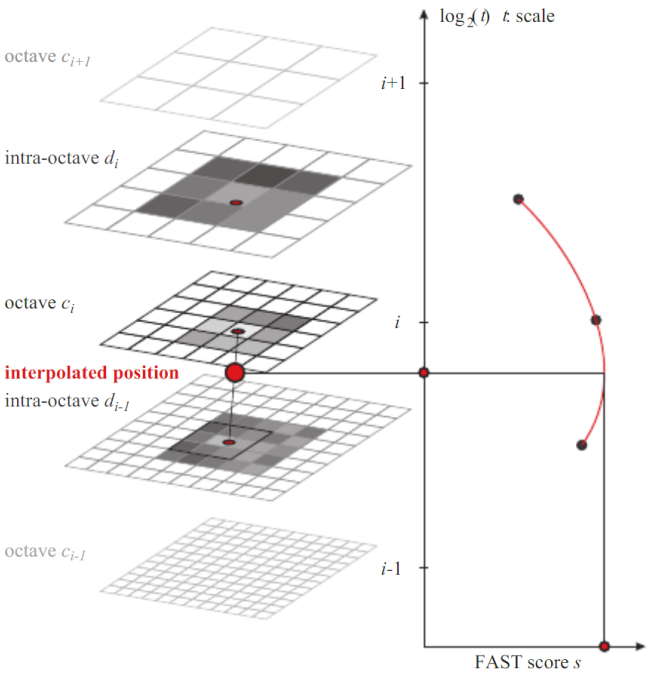
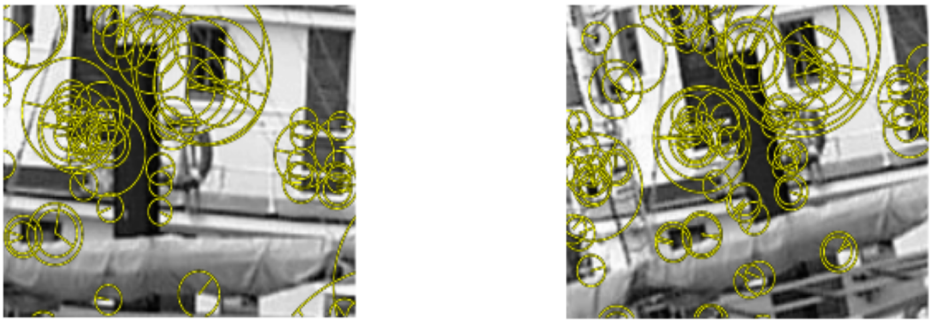



FIGURE 5 – Détection de points d'intérêt à l'espace échelle : Un point clé

Ensuite, pour affiner précisément la position du point clé, une amélioration est effectuée au niveau du sous-pixel. Enfin, une courbe parabolique à une dimension est ajustée le long de l'axe d'échelle pour déterminer l'échelle réelle du point clé. L'emplacement final du point clé est également ajusté entre les points les plus saillants dans la région, basés sur l'échelle déterminée.



(a) Boat image 1 (b) Boat image 2

FIGURE 6 – Gros plan d'un exemple de détection BRISK sur les images 1 et 2 de la séquence Boat présentant un petit zoom et une rotation dans le plan.

## 2.2 Description du point clé

Le descripteur BRISK est un outil clé pour la représentation efficace des points clés dans une image. Ces points clés sont définis par des emplacements précis (sous-pixel) et des valeurs d'échelle en virgule flottante. Le descripteur BRISK est formé en concaténant les résultats de tests simples de comparaison de luminosité, créant

ainsi une chaîne binaire distinctive pour chaque point clé.

L'une des caractéristiques notables de BRISK est l'identification de la direction caractéristique de chaque point clé. Cette information est utilisée pour normaliser les descripteurs en fonction de l'orientation, assurant ainsi une invariance de l'orientation. Cette invariance est cruciale pour la robustesse globale du descripteur, permettant de maintenir la performance même en cas de variations d'orientation.

De plus, dans le processus de construction du descripteur BRISK, une sélection minutieuse des comparaisons de luminosité est effectuée. Cette sélection est soigneusement optimisée pour maximiser la descriptivité des caractéristiques, assurant que le descripteur puisse capturer au mieux les informations distinctives des points clés dans l'image.

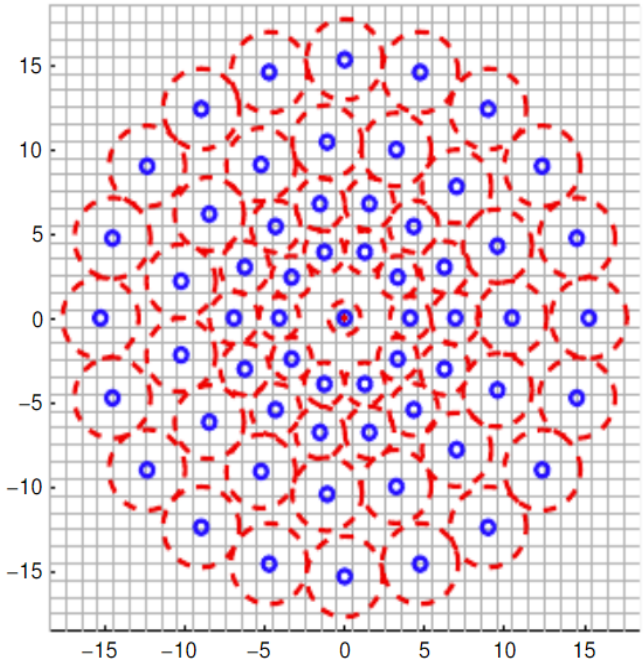



FIGURE 7 – Le modèle d'échantillonnage BRISK avec N=60 points : les petits cercles bleus indiquent les lieux d'échantillonnage ; les cercles plus grands, en pointillés rouges, sont tracés à un rayon $\sigma$ correspondant à l'écart-type du noyau gaussien utilisé pour lisser les valeurs d'intensité aux points d'échantillonnage.Le modèle présenté s'applique à une échelle de t=1.

### 2.2.1 Création du Descripteur BRISK : Modélisation, Échantillonnage et Construction du Descripteur Binaire

Le processus de création du descripteur BRISK se décompose en plusieurs étapes. Tout d'abord, un modèle d'échantillonnage de pixels lissés est défini autour de la caractéristique, impliquant l'utilisation d'un lissage gaussien pour prévenir les effets de repliement. Ensuite, les paires de pixels sont divisées en deux ensembles en fonction de leur distance, créant des ensembles de paires à courte et à longue distance. Le gradient local est calculé entre les paires à longue distance, et ces gradients sont sommés pour déterminer l'orientation globale de la caractéristique. Les

paires à courte distance sont ensuite ajustées en fonction de cette orientation. Enfin, un descripteur binaire est construit à partir des paires à courte distance ajustées, offrant une représentation distinctive et robuste de la caractéristique dans l'image. Ce processus permet au descripteur BRISK de capturer efficacement les informations significatives des points clés, tout en assurant une invariance à la rotation et une robustesse face aux variations d'échelle.

### 2.2.2 Correspondance des descripteur

Dans le processus de correspondance entre des points clés des images, la *"distance de Hamming"* est calculée pour toutes les paires de descripteurs entre les images. La distance de Hamming mesure le nombre de bits qui diffèrent entre deux chaînes de bits de même longueur. En l'occurrence, les descripteurs sont des chaînes de 512 bits. En comptant le nombre de bits correspondants entre les descripteurs, on évalue la similarité entre les points clés. Cette opération est efficace car elle se réduit simplement à une opération XOR (OU exclusif) entre les deux chaînes de bits. Le seuil de correspondance est déterminé en fonction du nombre de bits pouvant être appariés avec succès. Ainsi, si la distance de Hamming entre deux descripteurs est inférieure à ce seuil, les points clés correspondants sont considérés comme appariés, indiquant une correspondance significative entre les caractéristiques des deux images.

## 2.3 Répétabilité

Le score de répétabilité est une mesure évaluant la qualité des appariements de points clés entre deux images. Il est calculé en prenant le rapport entre le nombre de points clés correspondants et le nombre total minimum de points clés visibles dans les deux images. Pour chaque paire de points clés correspondants, un cercle proportionnel à l'échelle autour de chaque point clé sur une image est transformé sur l'autre image. Si le cercle et l'ellipse résultante se chevauchent de plus de 50% de l'union de leurs formes, cela indique une correspondance réussie. En d'autres termes, si une partie significative des régions entourant les points clés dans les deux images se recouvre, une correspondance est établie. Cette méthode utilise des cercles de taille comparable à ceux créés par d'autres méthodes telles que SIFT et SURF, permettant ainsi une évaluation juste et comparative de la répétabilité des points clés entre les images.

### 2.3.1 Exemple de correspondance BRISK

Un seuil de détection de 70 est utilisé et un seuil de distance de Hamming de 90. Les correspondances obtenues sont reliées par les lignes vertes qui ne montrent pas de faux positifs évidents.

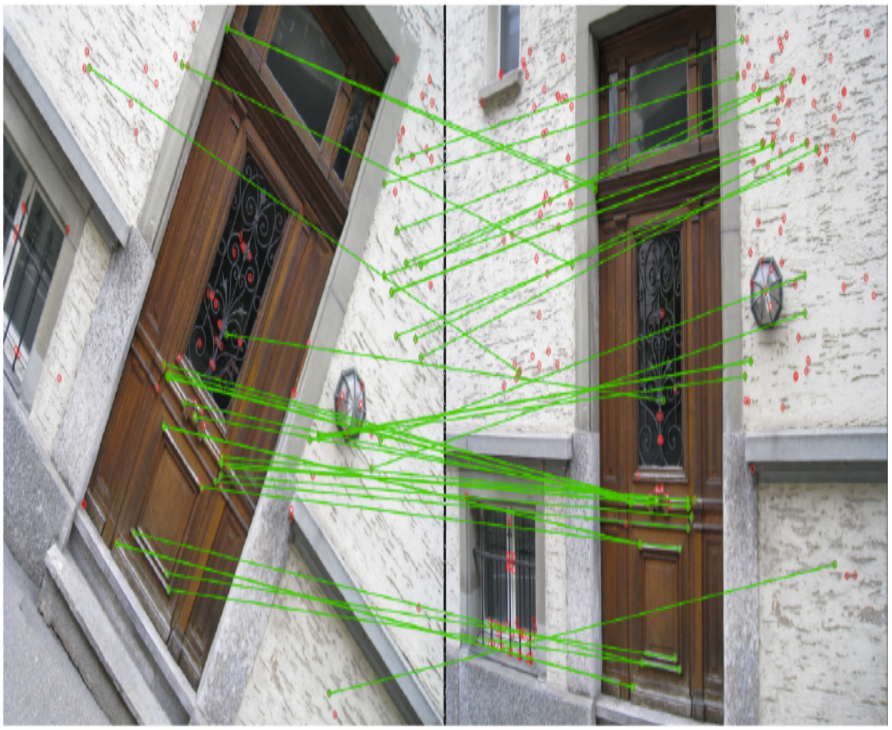


FIGURE 8 – Exemple de correspondance BRISK

# 3 KAZE

KAZE, acronyme de *"Keypoints Affine and Zoomed Ellipses"*, a été introduit en 2012 pour surmonter les limitations des espaces d'échelle linéaires. Cette approche repose sur des espaces d'échelle non linéaires construits avec des techniques avancées telles que l'Additive Operator Splitting (AOS) et la diffusion à conductance variable. KAZE se distingue par sa capacité à obtenir des caractéristiques présentant une répétabilité et une distinctivité élevées, même face à des transformations complexes d'images. En tant qu'avancée majeure dans la détection de caractéristiques, KAZE offre une adaptabilité accrue et des performances supérieures.

## 3.1 Modélisation de la Diffusion Non Linéaire : Concepts Fondamentaux et Approches Numériques

Cette section explore la modélisation de la diffusion non linéaire, une technique essentielle pour décrire l'évolution de la luminance à différentes échelles dans une image. Les approches reposent sur des équations différentielles partielles non linéaires (EDP) décrivant la divergence d'une fonction de flux qui guide la diffusion. La première partie introduit la formulation classique de la diffusion non linéaire, incluant une fonction de conductivité adaptative pour la diffusion en fonction de la structure locale de l'image. Elle détaille également l'équation de diffusion de Perona et Malik, ajustant la conductivité en fonction du gradient pour réduire la diffusion aux bords. La seconde partie aborde le schéma de discrétisation "Additive Operator Splitting" (AOS), utilisé pour approximer numériquement les EDP non linéaires. Ce

volet couvre les méthodes de discrétisation, y compris le schéma linéaire-implicite et son application à la diffusion. En résumé, cette section offre une compréhension approfondie des concepts fondamentaux et des approches numériques liés à la modélisation de la diffusion non linéaire.

## 3.2 Caracteristiques de KAZE

### 3.2.1 Calcul de l'espace d'échelle non linéaire

Dans cette section dédiée aux caractéristiques de KAZE, l'accent est mis sur la méthodologie qu'utilise KAZE pour calculer son espace d'échelle non linéaire. La stratégie adoptée par KAZE, similaire à SIFT mais avec des nuances distinctives, se distingue notamment par son choix de ne pas sous-échantillonner entre les différentes octaves. Contrairement à SIFT, qui travaille avec des images sous-échantillonnées pour chaque nouvelle octave, KAZE conserve la résolution de l'image originale. Les indices discrets d'octave et de sous-niveau sont exploités pour identifier les différentes échelles, tandis qu'une conversion des niveaux d'échelle discrets en unités de temps est introduite pour aligner le filtre de diffusion non linéaire. L'utilisation de schémas AOS offre une stabilité pour construire itérativement l'espace d'échelle non linéaire de KAZE. Les visualisations comparatives entre l'espace d'échelle de diffusion gaussien et non linéaire ajoutent une dimension visuelle, soulignant l'efficacité de l'approche non linéaire dans la capture de structures à différentes échelles temporelles.

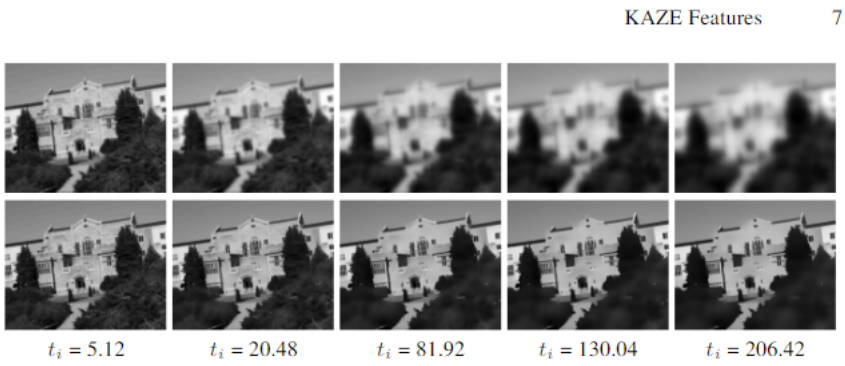



FIGURE 9 – Comparaison entre l'espace d'échelle de diffusion gaussien et non linéaire pour plusieurs temps d'évolution ti

Premier rang : espace d'échelle gaussien (diffusion linéaire). L'espace d'échelle est formé en transformant l'image originale avec un noyau gaussien d'écart-type croissant.
Deuxième rangée : espace d'échelle de diffusion non linéaire avec la fonction de conductivité g3.

## 3.3 Détection des caractéristiques

Pour détecter les points d'intérêt, la méthode utilise le calcul de la réponse du déterminant du hessien normalisé à plusieurs niveaux d'échelle. La détection de caractéristiques à différentes échelles nécessite la normalisation de l'ensemble des opérateurs différentiels en fonction de l'échelle, car l'amplitude des dérivées spatiales diminue généralement avec l'échelle. Les dérivées de premier et de second ordre sont estimées en utilisant des filtres Scharr de dimensions $3 \times 3$ avec différentes tailles de pas de dérivée $\sigma_i$. Les dérivées de second ordre sont approximées en utilisant des filtres de Scharr consécutifs dans les coordonnées souhaitées de la dérivée. Cette approche permet de calculer une réponse robuste du déterminant du hessien normalisé, qui est essentielle pour la détection fiable de points d'intérêt, en tenant compte de la variabilité d'échelle dans l'image.

## 3.4 Construction du Descripteur KAZE

### 3.4.1 Estimation de l'Orientation et Détection des Points Clés

Initialement, pour assurer l'invariance à la rotation, il est essentiel d'estimer l'orientation dominante autour de chaque point clé. Cette estimation s'effectue dans un voisinage local du point clé en calculant les dérivées de premier et de second ordre sur l'espace d'échelle non linéaire. Les points clés sont ensuite détectés par la recherche des extrema dans cet espace d'échelle. L'orientation dominante est déterminée en analysant les réponses aux dérivées dans un voisinage circulaire du point clé.

### 3.4.2 Construction du Descripteur

La construction du descripteur proprement dite comporte plusieurs phases. Tout d'abord, un voisinage rectangulaire autour du point clé est défini. Ce voisinage est subdivisé en sous-régions, et les réponses dérivées dans chaque sous-région sont pondérées par une gaussienne centrée sur le centre de la sous-région. Ces réponses pondérées sont ensuite agrégées dans un vecteur descripteur, représentant ainsi les caractéristiques locales de l'image autour du point clé.

Enfin, pour garantir une certaine invariance, le vecteur descripteur est normalisé. Cette normalisation contribue à rendre le descripteur KAZE indépendant du contraste, améliorant ainsi sa robustesse face à des variations d'éclairement.

## Deuxième partie

# Apprentissage

L'approche par apprentissage est une technique supervisée qui vise à mettre en place une fonction de similarité mesurant à quel point deux objets se ressemblent. Elle repose sur la formation d'un modèle avec un ensemble de données d'entraînement pour prédire la similarité entre deux images. Deux images sont prises en entrée et une sortie est produite, la sortie indique si les images sont similaires ou non.

La méthode par apprentissage peut être utiliser pour la problématique de l'image matching et la problématique de la recapture. Pour l'image matching, nous utiliserons un réseau siamois dans lequel nous donnerons en entrée une image normale et une autre image avec diverses légères altérations et il faudra que l'algorithme détermine s'ils sont approximativement similaires. Pour la recapture, il faut détecter le rendu d'une image avec une simulation de LED (Light-Emitting Diode) RGB (Red, Green, Blue) qui simule la recapture sur un écran pour cela on utilise un réseau de classification binaire.

## 1 Architecture

### 1.1 Réseau de neurones

Un réseau de neuronnes standard possède une couche d'entrée, une couche de sortie et potentiellement d'autres couches cachées entre elles. Ces couches sont composées de neurones connectés entre eux, ces neurones sont des unités de traitement de données qui permettent l'apprentissage du réseau (voir figure 10).

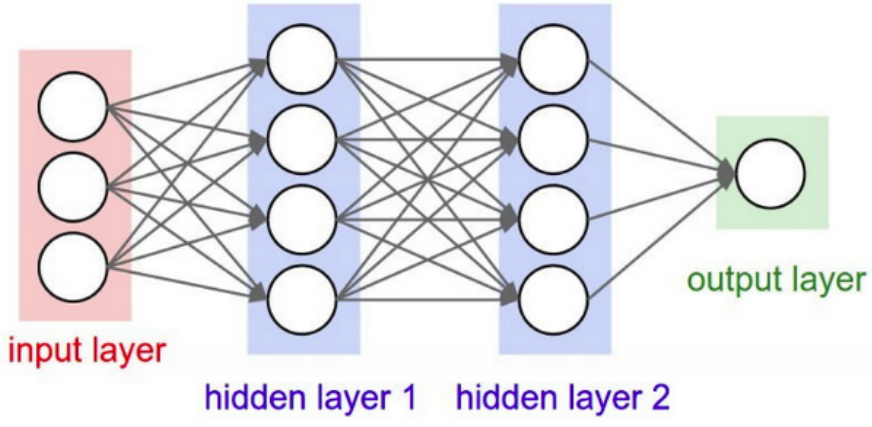



Figure 10 – Un réseau [9]

Chaque connexion entre les neurones est associée à un poids qui est ajusté

pendant la phase d'apprentissage du réseau, ces poids influencent l'entraînement du modèle. Des fonctions de non-linéarités sont introduites dans un réseau pour permettre au réseau de mieux s'adapter et s'entraîner avec des données plus complexes.

Un biais est ajouté aux entrées d'un réseau pour aussi permettre au réseau de mieux s'adapter et d'ajouter de la flexibilité.

### 1.2 Classificateur

Pour la classification, il faut séparer les données d'entrées en deux types pour le réseau : des données d'entrées et des étiquettes qui indiquent la sortie voulu pour chaque donnée d'entrée. Ces étiquettes sont égales à 0 ou 1.

Par exemple pour le cas de la recapture, dans le dataset que nous utiliserons, nous aurons des images normales avec une étiquette égal à 0 et des images avec simulation de LED qui ont une étiquette égal à 1. Après avoir entraîné le réseau, grâce aux étiquettes qui indiquent la sortie voulu, le réseau doit pouvoir déterminer (avec des données d'entrées différentes) quelles images ont une simulation de LED et quelles images n'ont pas de simulation de LED.

## 2 Réseau siamois

Une technique permettant de déterminer la similarité des entrées est le réseau siamois. Le réseau siamois est souvent utilisé pour la comparaison d'images. Il utilise une architecture composée de deux sous-réseaux identiques pour comparer les caractéristiques extraites de deux images d'entrée. Il mesure la similarité entre ces images en calculant la distance euclidienne entre les vecteurs des caractéristiques extraites.

Pour alimenter l'algorithme, il faut générer des images avec différentes altérations : des assiettes différentes, des zooms, etc. Les images générées seront utilisées pour apprendre au réseau siamois la comparaison d'images. Le réseau sera ensuite testé avec des nouvelles images que l'on n'a pas utilisées pour l'apprentissage.

### 2.1 Partage des paramètres

Les deux sous-réseaux du réseau siamois sont identiques du point de vue de leur architecture, elles ont le même nombre de couches et le même nombre de neurones par couche. Chaque couche dans une branche a une correspondance directe avec la même couche dans l'autre branche.

Donc, les poids et les biais de chaque couche d'une branche sont exactement les mêmes que ceux de la couche correspondante dans l'autre branche. Cela signifie que les transformations effectuées sur les données par les deux réseaux sont identiques.

En partageant les paramètres, les deux réseaux sont forcés de passer par le même apprentissage et d'apprendre les mêmes caractéristiques, cela permet de ne pas ajuster indépendamment des paramètres spécifiques à chaque réseau.

Par rapport à un réseau standard, l'entraînement est aussi amélioré par le partage des poids. Le réseau siamois est moins susceptible de mal ajuster les données d'entraînement, car il apprend des caractéristiques générales des deux réseaux plutôt que des caractéristiques spécifiques à chaque réseau.

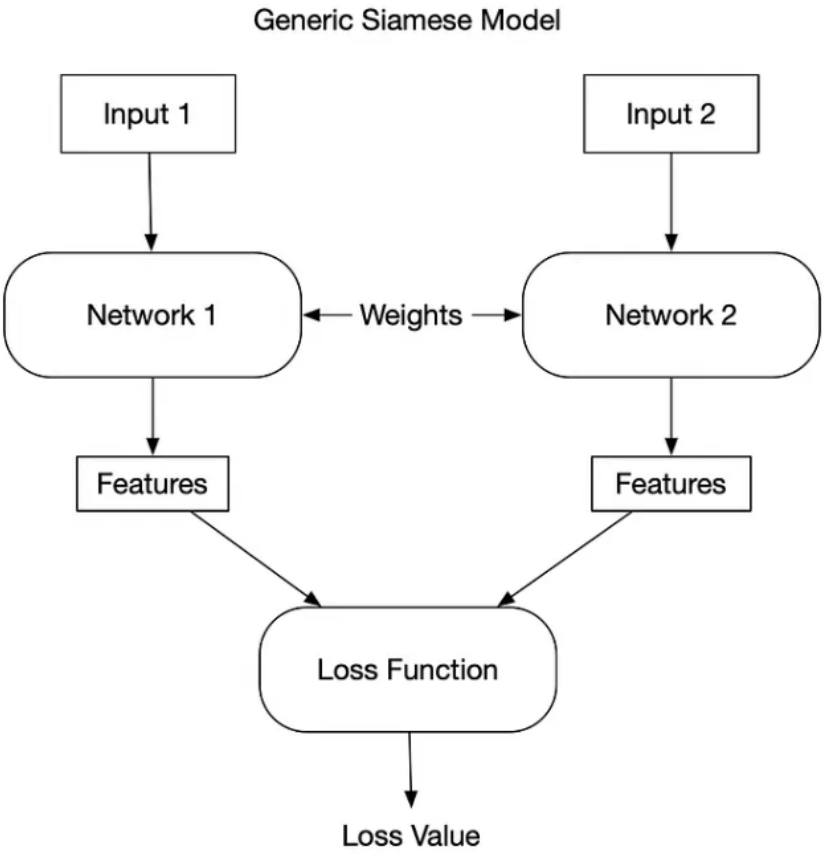



FIGURE 11 – Réseau siamois [3]

## 2.2 Fonction de perte

La fonction de perte d'un réseau siamois est conçue pour mesurer à quel point les sorties générées par les deux branches du réseau sont similaires ou différentes, en fonction des étiquettes associées aux paires d'entrées.

Un calcul de similarité se trouve dans la fonction de perte. Cela permet de mesurer la similitude entre les paires d'entrées. Les deux sous-réseaux génèrent des vecteurs pour chaque entrée en passant à travers leurs couches respectives. Ces vecteurs contiennent les caractéristiques importantes des entrées. La distance euclidienne est calculé entre ces deux vecteurs pour mesurer leurs différences.

Ensuite, nous choisissons un seuil qui détermine la distance minimale de simi-

larité voulue entre les deux vecteurs des deux entrés. Mais ce seuil ne doit pas être trop bas parce que si la distance minimale de similarité est trop basse, il sera difficile pour le réseau siamois de différencier les entrées qui sont similaires et ceux qui ne le sont pas.

## Troisième partie

# Recapture

Avec les progrès de la technologie d'affichage des images et l'utilisation généralisée de divers dispositifs d'acquisition d'images, il devient très pratique et facile de recapturer des images de haute qualité à partir d'écrans LCD (affichage à cristaux liquides) de haute fidélité. Ces images recapturées constituent une menace sérieuse pour les technologies actuelles de criminalistique d'images et les systèmes de reconnaissance intelligents. Les applications critiques telles que l'identification biométrique des personnes (par exemple, la reconnaissance faciale) sont confrontées à des problèmes d'usurpation de visage, le système pouvant être trompé en montrant la photo d'une autre personne sur un écran électronique. Ce rapport vise à fournir un aperçu de deux solutions de reconnaissance de recapture, une par apprentissage de classificateur SVM et l'autre par analyse de descripteurs LTC. Le cadre applicatif ne se restreignant pas seulement à la forensique, ces propositions seront implémentées dans un jeu mobile, invitant les joueurs à capturer des images et à défier leurs contacts à les recréer avec précision.

## 1 Méthode par apprentissage de coefficients de corrélation avec classificateur SVM

Lors de la recapture, une perte de netteté est généralement subie, entraînant un lissage de l'image, et des distorsions visuelles dans des détails fins. La première solution [1] consiste en une comparaison des pixels des images originale et recapturée afin d'y retrouver ces différences générées par la recapture.

Pour chacune des 2 images originale et recapturée, on cherche d'abord à obtenir leurs versions résiduelles $R1$ et $R2$, chaque version étant respectivement la différence entre l'image originale et celle filtrée par un filtre passe-bas simple. Pour un pixel $X_j$ de l'image originale, le pixel correspondant dans $R1$ (respectivement $R2$) représente la différence de valeurs de niveaux de gris entre $X_j$ et la valeur moyenne de ses deux pixels voisins horizontaux (respectivement verticaux). Les deux images ainsi obtenues décrivent les bords et le bruit de l'image, mettant en avant les altérations de flou et anti-aliasing. Ces altérations n'ayant pas de liens avec la couleur des pixels, les images originales sont directement converties en niveaux de gris.

Chaque image résiduelle est ensuite découpée en un chevauchement de petites

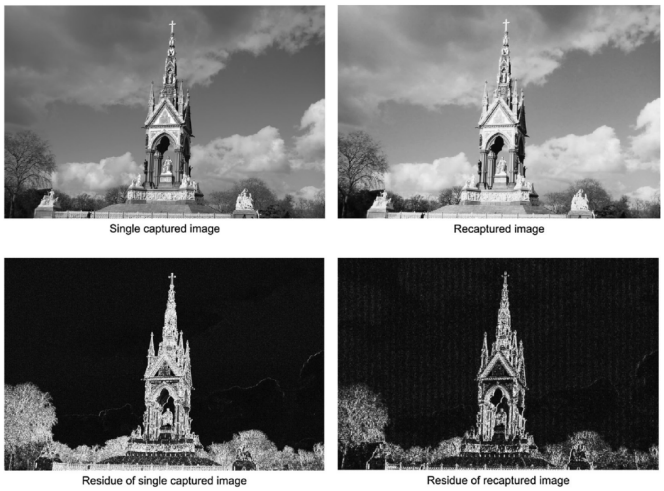



FIGURE 12 – Dans la rangée supérieure, une paire d'images capturées et recapturées en niveaux de gris. Dans la rangée du bas, leur version résiduelle.

régions, ou patchs, de 5x5 pixels. Les éléments de de chaque patch $P^{(k)}$ sont notés $p_{i;j}^{(k)}$ avec $i, j \in \{1, 2, 3, 4, 5\}$ et $k \in \{1, 2, \ldots, k\}$. On rassemble chaque pixel de chaque patch de même coordonnées dans un vecteur $V_{i,j}$, avec i et j coordonnées du patch. On calcule ensuite le coefficient de corrélation $c_{i,j}$ (horizontal pour $R1$ et vertical pour $R2$) entre le vecteur correspondant au pixel central de chaque patch $V_{3,3}$ et les 25 autres vecteurs :

$$c_{i,j} = \rho_{V_{3,3},V_{i,j}} = \frac{\sum_{k=1}^{K} \left(p_{3;3}^{(k)} - \overline{p(3,3)}\right)\left(p_{i;j}^{(k)} - \overline{p(i,j)}\right)}{\sqrt{\sum_{k=1}^{K} \left(p_{3;3}^{(k)} - \overline{p(3,3)}\right)^2} \sqrt{\sum_{k=1}^{K} \left(p_{i;j}^{(k)} - \overline{p(i,j)}\right)^2}} \tag{1}$$

Où $p_{3;3}$ et $p_{i;j}$ sont les moyennes des éléments dans $V_{3,3}$ et $V_{i,j}$, et où $i, j \in \{1, 2, 3, 4, 5\}$. On obtient deux matrice de coefficients de corrélation horizontaux et verticaux de taille 5x5. Il existe une forte symétrie entre les deux parties séparées par la diagonale majeure des matrices, on choisit donc de ne garder que le triangle supérieur pour éviter la redondance, et on enlève le coefficient central, toujours égal à 1. Les valeurs des coefficients sont nettement plus élevées dans la matrice de l'image originale que dans la matrice de l'image recapturée : cette différence implique une forte capacité de discrimination.

Finalement, chaque matrice ( Deux pour chaque image résiduelle) est utilisée pour entraîner un classificateur à base de machines de vecteurs à support (SVM), qui servira d'outil de prédiction pour les futures images à comparer. Le classificateur SVM utilisé est un composant prenant en entrée une matrice de coefficients de corrélation et renvoie 0 s'il considère l'image comme étant originale, et 1 sinon. L'entraînement consiste à labelliser les matrices d'images originales et recapturées et de les fournir au classificateur.

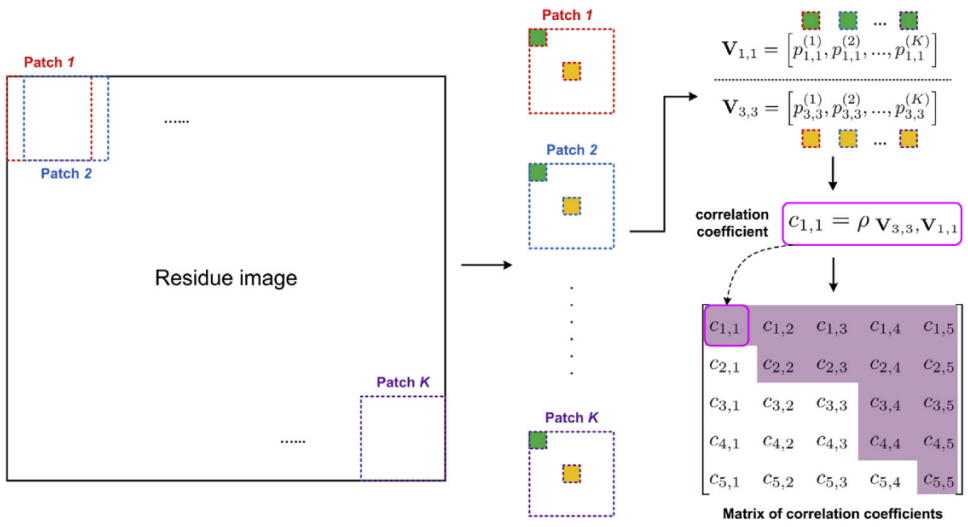



FIGURE 13 – Extraction des patchs et calcul des coefficients de corrélation par pixel, en prenant $c_{1,1}$ comme exemple. On conserve le triangle supérieur de la matrice de coefficients.

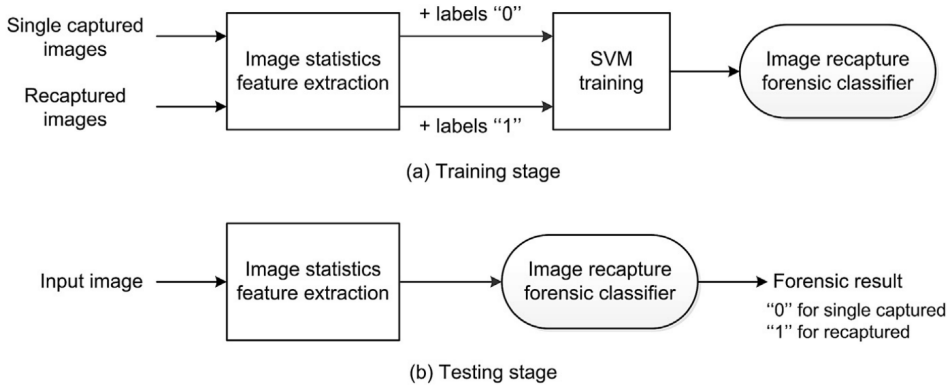



FIGURE 14 – Schéma fonctionnel (a) de la phase d'entraînement et (b) de la phase d'essai.

# 2 Méthode par apprentissage de descripteurs LTC avec classificateur neuronal

La deuxième méthode [2] se concentre sur les altérations de couleur et de luminosité qui se produisent lors de la recapture. Les caméras grand public utilisent un réseau de filtres colorés (CFA) pour capturer différentes composantes de couleur, ou chaque pixel du filtre capture la couleur qui lui est attribuée (rouge, vert ou bleu). Des algorithmes d'interpolation permettent ensuite d'estimer les couleurs manquantes et d'obtenir les trois canaux de couleur complets. Cela introduit des corrélations entre ces canaux, qui peuvent changer lors de la recapture de l'image.

Pour mieux capturer ces différences, l'image traitée est convertie de l'espace couleur RVB à l'espace couleur HSV (Teinte (H), Saturation et Valeur), l'espace HSV étant plus adapté pour représenter les variations de luminosité et de couleur. L'image ainsi définie permettra une capture des corrélations entre pixels à petite échelle. Afin de capturer les corrélations de plus grande échelle, une image moyenne

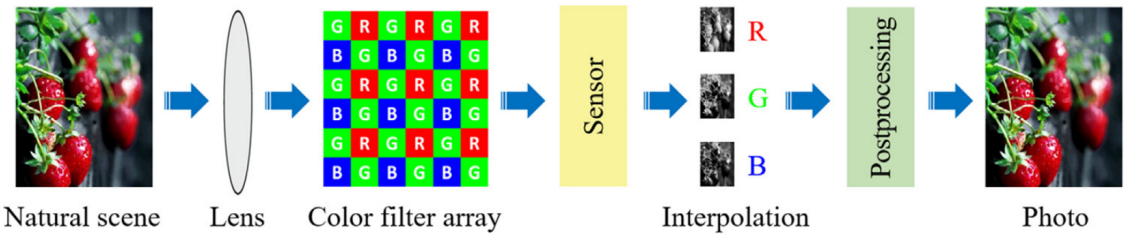



FIGURE 15 – Procédé d'imagerie d'un appareil photo grand public

groupée $I^*$ est obtenue en prenant la somme des valeurs de pixels de l'image d'origine $I$ sur des régions de taille $2 * 2$ :

$$I^*(x, y, z) = \sum_{u=0}^{1} \sum_{v=0}^{1} \frac{I(2x - u, 2y - v, z)}{4} \tag{2}$$

Où $x \in \{1, 2, \ldots, \lfloor \frac{M}{2} \rfloor\}$, $y \in \{1, 2, \ldots, \lfloor \frac{N}{2} \rfloor\}$, $z \in \{1, 2, 3\}$, et $\lfloor \cdot \rfloor$ représente la partie entière.

Sont ensuite obtenues des cartes de différences du second ordre, représentant la valeur de différence entre un pixel et ses voisins pour une direction (horizontale, verticale, diagonale) et une caractéristique (H, S, V) donnés. En prenant pour exemple le canal de teinte de $I$ noté $I_H$ et $I_H(i, j)$ la valeur d'un pixel à la position $(i, j)$, la carte différencielle du second ordre dans la direction horizontale est définie comme suit :

$$D_{I_H}^{(h)} = 2I_H(i, j) - I_H(i, j - 1) - I_H(i, j + 1) \tag{3}$$

De la même manière que pour l'image originale, on obtient les cartes de différence du second ordre de l'image moyenne groupée.

Ces cartes de différences peuvent refléter les corrélations entre pixels d'un même direction, mais négligent les corrélations entre des directions différentes. Pour ajouter une dimension supplémentaire, on utilise des cartes d'erreur de prédiction : des prédicteurs linéaires tentent de prédire la valeur d'un pixel en fonction de ses voisins dans différentes directions. En utilisant la carte de différence $I_H$ comme exemple, le prédicteur linéaire pour la magnitude des cartes de second ordre est donné par :

$$|D_{I_H}^{(h)}| = w_1|D_{I_H}^{(v)}| + w_2|D_{I_H}^{(d)}| + w_3|D_{I_H}^{(c)}| \tag{4}$$

Où $w_k$ sont les poids et $|\cdot|$ représente la valeur absolue. On obtient la carte d'erreur de prédiction $P_{I_H}^{(h)}$ de la manière suivante :

$$P_{I_H}^{(h)} = \log_2(V_h) - \log_2(|Q_h \vec{w}|) \tag{5}$$

Où $V_h$ est la matrice colonne contenant les valeurs de $D_{I_H}^{(h)}$, $Q_h$ une matrice de 3 colonnes contenant $D_{I_H}^{(v)}$, $D_{I_H}^{(d)}$ et $D_{I_H}^{(c)}$ avec v, d et c représentant les 3 autres directions

(verticale, diagonale et contre-diagonale), et $\vec{w} = (w_1, w_2, w_3)^T$. On effectue le même calcul pour obtenir les cartes d'erreur de prédiction de l'image moyenne groupée.

Finalement, un histogramme LTC (light ternary count) normalisé est calculé à partir des cartes d'erreur de prédiction. Pour l'obtenir, on utilise un modèle ternaire local (LTP). On divise la carte d'erreur de prédiction en régions de taille $3 * 3$, puis on applique la fonction $f(E_i, E_c, t)$ à 3 paramètres :

$$f(E_i, E_c, t) = \begin{cases} +1, & E_i > E_c + t \\ 0, & |E_i - E_c| \leq t \\ -1, & E_i < E_c - t \end{cases} \tag{6}$$

Où $E_c$ est le pixel central du patch et $E_i$ ses pixels voisins. $t$ est un seuil spécifié par l'utilisateur, fixé à 2 dans cet exemple. On obtient une matrice, ou motif ternaire, ensuite divisée en 2 matrices ou motifs inférieurs et supérieurs. les codes binaires des motifs supérieur et inférieur sont cumulés pour obtenir les codes LTC supérieur et inférieur, respectivement.

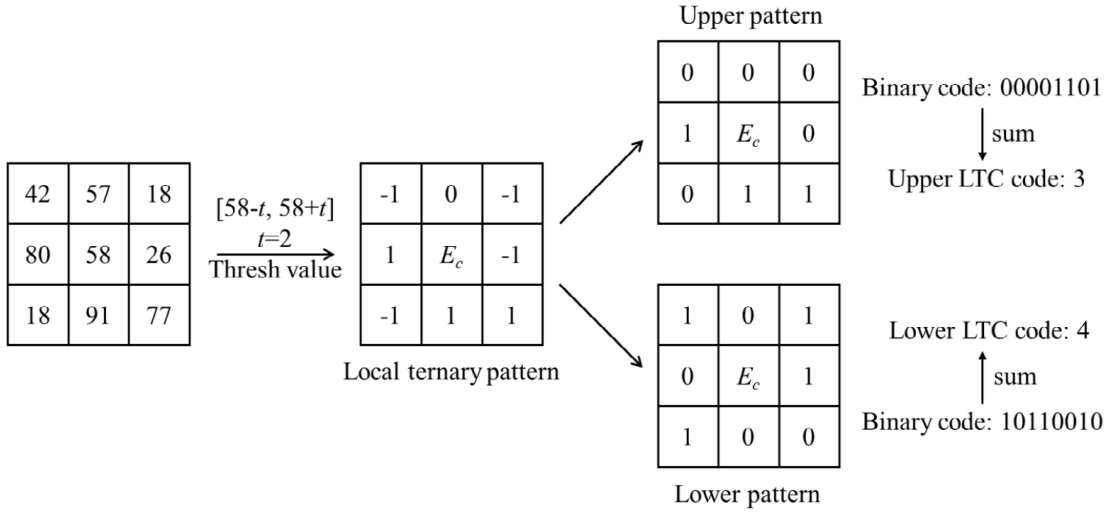



FIGURE 16 – Calcul du code LTC

L'histogramme LTC normalisé $H(k)$ pour l'ensemble des motifs inférieurs $F_l$ et supérieurs $F_u$ d'une carte d'erreur de prédiction $P^{(*)}$ de taille $U * V$ est donné par :

$$H(k) = \frac{1}{U * V} \sum_{p=1}^{U} \sum_{q=1}^{V} \ell(F_*(p, q), k) \tag{7}$$

Où $k \in \{0, 1, \ldots, 8\}$ et $* \in \{u, l\}$. La fonction $l()$ produit 1 si ses deux paramètres sont égaux, sinon elle produit 0. Enfin, la caractéristique LTC est obtenue à partir des deux histogrammes LTC de l'image originale et moyenne groupée.

De la même manière que dans la méthode précédente, on entraîne un réseau neuronal avec en entrée les caractéristiques LTC labellisées et en sortie une réponse binaire, 0 pour une image originale et 1 pour une recapturée.

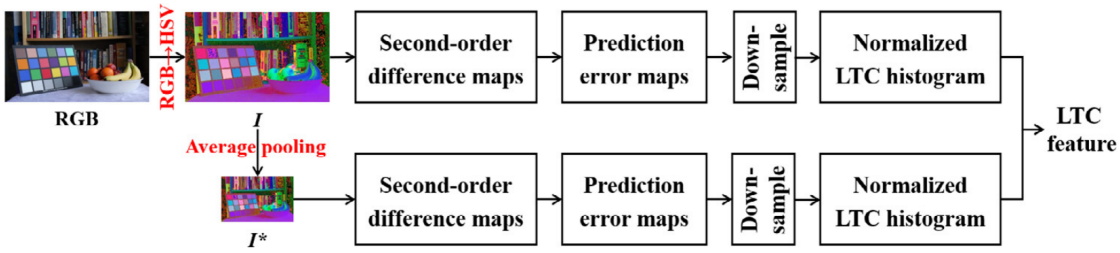



FIGURE 17 – Schéma du processus d'extraction des caractéristiques LTC

# Conclusion

En conlusion, pour l'image matching nous avons choisi de travailler en particulier sur BRISK et KAZE, et nous considérons que KAZE est le meilleur algorithme, parce que selon nos recherches et par exemple d'après cette étude [8] nous voyons qu'il met en évidence les performances remarquables de KAZE (ainsi que de BRISK) dans certaines conditions. Ces algorithmes ont démontré une robustesse significative en terme d'invariance à l'échelle et à la rotation. Pour enrichir notre analyse, nous avons inclus les algorithmes SIFT, BRISK, KAZE et ORB. La justification de notre choix répose sur les résultats de l'étude citée ([8] où BRISK et KAZE se sont démarqués par leur capacité à maintenir une performance élevée dans des contextes variés. ORB quant à lui est reputé pour sa rapidité. Au second semestre, il faudra faire des comparaisons avec SIFT, BRISK et KAZE.

Avec l'approche par apprentissage, nous avons besoin de plusieurs datasets d'image pour entraîner et tester les algorithmes. Pour l'image matching, il faudra générer des images avec différentes altérations : des assiettes différentes, des zooms, etc. Ces images alimenteront le réseau siamois et seront comparer avec les mêmes images mais sans altérations pour entraîner le réseau. Pour la recapture il faudra générer des images avec des simulations de LED. Ces images altérées et des images non altérées seront classées (classe 0 ou 1) et alimenteront un classificateur pour qu'il s'entraîne à prédire la classe d'une image, c'est-à-dire si l'image est une recapture ou non.

Pour la recapture, une comparaison sur différents ensembles d'images de l'ensemble des méthodes les plus utilisées est explicitée dans [2]. La seconde solution par par apprentissage de descripteurs LTC avec classificateur neuronal présente des résultats similaires, parfois meilleurs que la méthode par apprentissage de coefficients de corrélation avec classificateur SVM pour un ensemble d'images donnés. Pour un ensemble mixte composé de plusieurs ensembles d'images, les résultats de la seconde méthode sont meilleurs que la première. En sachant que notre projet comportera des images provenant d'appareils de capture différents et donc plus en accord avec ces derniers résultats, le choix d'implémentation se porte donc de manière évidente sur cette dernière.

# Références


[1] A simple and effective image-statistics-based approach to detecting recaptured images from LCD screens. *Digital Investigation*, 2017. https://doi.org/10.1016/j.diin.2017.10.001

[2] Recaptured image forensics based on local ternary count of high order prediction error. *Image and Vision Computing*, 2022. https://doi.org/10.1016/j.image.2022.116662

[3] A Comprehensive Guide to Siamese Neural Networks. https://medium.com/@rinkinag24/a-comprehensive-guide-to-siamese-neural-networks-3358658c0513

[4] Distinctive Image Features from Scale-Invariant Keypoints - DAVID G. LOWE https://doi.org/10.1023/B%3AVISI.0000029664.99615.94

[5] Introduction to SIFT( Scale Invariant Feature Transform) https://medium.com/@deepanshut041/introduction-to-sift-scale-invariant-feature-transform-65d7f3a72d40

[6] Leutenegger S., Chli M., & Siegwart R. Y. (2011). BRISK : Binary robust invariant scalable keypoints. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. https://www.researchgate.net/publication/221110715_BRISK_Binary_Robust_invariant_scalable_keypoints

[7] Alcantarilla P. F., Bartoli A., & Davison A. J. (2012). KAZE Features. *European Conference on Computer Vision (ECCV)*. https://www.researchgate.net/publication/236985005_KAZE_Features

[8] A comparative analysis of SIFT, SURF, KAZE, AKAZE, ORB, and BRISK. https://doi.org/10.1109/ICOMET.2018.8346440

[9] Présentation des réseaux de neurones artificiels. https://blog.sinatechnologie.com/presentation-des-reseaux-de-neurones-artificiels.